TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG (DAU)

Chương 3: Cấu trúc dữ liệu tập hợp

Bài giảng · Chương 3

Học thật · làm thật · code thật

TS. Đỗ Phúc Hảo – Giảng viên *Lecturer*

Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng

haodp.ai@gmail.com · 0365 674 592 · Đà Nẵng, 2026

Nội dung buổi học *Lecture outline*

1. **Tổng quan Bộ khung** *framework*
2. **Danh sách** *List*
3. **Hàng đợi** *Queue & Deque*
4. **Tập hợp** *Set*
5. **Ánh xạ** *Map*
6. **equals & hashCode**
7. **Comparable & Comparator**
8. **Chọn cấu trúc & Big-O**

Mục tiêu: biết với mỗi tình huống thì chọn cấu trúc nào cho vừa đúng vừa nhanh. Slide tiếng Việt là chính; code Java comment tiếng Anh.



1

Bộ khung tập hợp

Collections Framework

Vì sao cần Bộ khung tập hợp *Why a framework*

DAU

ĐỊNH NGHĨA · DEFINITION

Bộ khung tập hợp *Collections Framework* là kiến trúc thống nhất gồm các giao tiếp mô tả kiểu tập hợp, các lớp cài đặt sẵn và thuật toán dùng chung.

Mảng: kích thước cố định, thiếu sẵn thêm/xoá/tìm.

Tập hợp: co giãn, giàu thao tác, đổi cài đặt dễ dàng.

- Tách giao tiếp khỏi cài đặt: lập trình theo giao tiếp.
- Thay lớp cài đặt mà ít phải sửa mã.

Hai nhánh lớn *Two separate roots*

DAU

Nhánh `Collection`
nhóm phần tử đơn lẻ.
`List`, `Set`, `Queue` kế thừa từ đây.

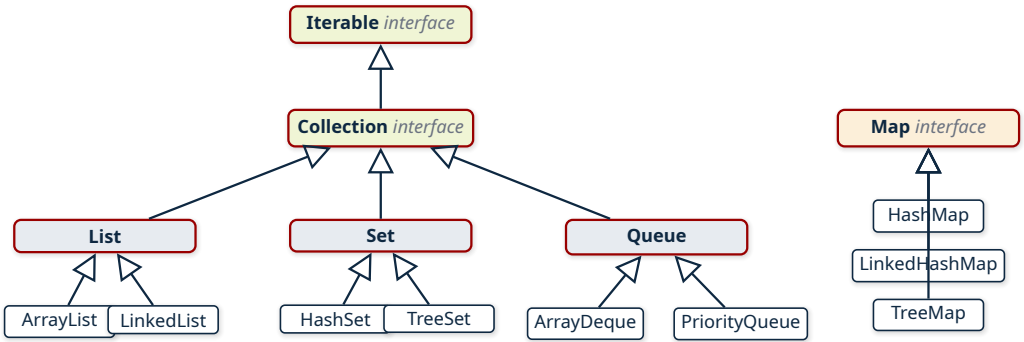
Nhánh `Map`
ánh xạ khoá → giá trị.
`Map` KHÔNG là `Collection`.

LỖI HAY GẶP · PITFALL

Điểm hay nhầm: `Map` thuộc Bộ khung tập hợp nhưng bản chất là cặp khoá-giá trị, không phải phần tử đơn, nên không kế thừa `Collection`.

Cây phân cấp Bộ khung tập hợp *Hierarchy*

DAU



Mũi tên tam giác rỗng đọc là kế thừa hoặc cài đặt giao tiếp.

Lặp với Iterable và Iterator *Iterating safely*

DAU

```
List<String> ten = new ArrayList<>(
      List.of("An", "Binh", "Cuong"));
for (String s : ten) { // for-each calls iterator()
   System.out.println(s);
}
Iterator<String> it = ten.iterator();
while (it.hasNext()) {
   String s = it.next();
   if (s.startsWith("B"))
      it.remove(); // safe removal while iterating
}
```

GHI NHỚ · KEY POINT

`for-each` chỉ là cú pháp đẹp gọi ngầm `iterator()`.

LỖI HAY GẶP · PITFALL

Xoá bằng `list.remove` trong `for-each` ném ngoại lệ `ConcurrentModificationException`. Dùng `it.remove()`.



2

Danh sách

ArrayList vs LinkedList

ArrayList so với LinkedList *Array vs linked*

DAU

`ArrayList` mảng động.
Truy cập chỉ số $O(1)$.
Chèn/xoá giữa $O(n)$.
Tận dụng tốt bộ nhớ đệm.

`LinkedList` liên kết kép.
Thêm/xoá hai đầu $O(1)$.
Truy cập phần tử thứ i: $O(n)$.
Phải đi lần từ đầu.

GHI NHỚ · KEY POINT

Thực tế `ArrayList` thắng ở đa số trường hợp. Chỉ chọn `LinkedList` khi thao tác chủ yếu là thêm hoặc xoá ở hai đầu.

Thao tác cơ bản trên List *List operations*

DAU

```java
List<String> ds = new ArrayList<>(); // program to the List interface
ds.add("An"); // append, amortized O(1)
ds.add("Binh");
ds.add(1, "Cuong"); // insert at index 1, O(n)
String x = ds.get(0); // random access by index, O(1)
ds.remove("Binh"); // remove by value, O(n)
System.out.println(ds.size()); // number of elements

List<Integer> dq = new LinkedList<>(); // choose when adding at both ends
((LinkedList<Integer>) dq).addFirst(1);
((LinkedList<Integer>) dq).addLast(2);
```



3

Hàng đợi

Queue & Deque

Queue, Deque, ArrayDeque *FIFO and two ends*

DAU

Queue: hàng đợi, vào trước ra trước *FIFO*.

Deque: hai đầu, dùng được cả ngăn xếp lẫn hàng đợi.

ArrayDeque cài `Deque` bằng mảng vòng, hai đầu $O(1)$.

Khuyến nghị thay lớp `Stack` cũ; thường nhanh hơn `LinkedList`.

Phương thức chính: `offer`/`poll` cho hàng đợi, `push`/`pop` khi dùng như ngăn xếp.

ArrayDeque: hàng đợi và ngăn xếp *Queue and stack*

DAU

```java
Deque<String> hangCho = new ArrayDeque<>();
hangCho.offer("job1"); // enqueue at tail, O(1)
hangCho.offer("job2");
String dau = hangCho.poll(); // dequeue from head, O(1) -> "job1"

Deque<Integer> nganXep = new ArrayDeque<>();
nganXep.push(10); // push onto stack top
nganXep.push(20);
int dinh = nganXep.pop(); // pop from stack top -> 20
```

PriorityQueue: lấy nhỏ nhất trước *Min-heap*

DAU

```java
PriorityQueue<Integer> pq =
      new PriorityQueue<>(); // min-heap, natural
          order
pq.offer(5); pq.offer(1); pq.offer(3);
while (!pq.isEmpty()) {
   System.out.print(pq.poll() + " "); // 1 3 5
}
// reverse into a max-heap
PriorityQueue<Integer> maxHeap =
   new PriorityQueue<>(Comparator.reverseOrder());
```

GHI NHỚ · KEY POINT

Đống nhị phân *binary heap*: thêm và lấy nhỏ nhất $O(\log n)$, xem nhỏ nhất $O(1)$.

Thứ tự ưu tiên lấy từ `Comparable` hoặc `Comparator` truyền vào.



4

Tập hợp

HashSet, LinkedHashSet, TreeSet

Ba kiểu Set *No duplicates*

DAU

`HashSet`
bảng băm, $O(1)$,
thứ tự không xác định.

`LinkedHashSet`
giữ thứ tự chèn,
chi phí cao hơn chút.

`TreeSet`
cây đỏ đen, $O(\log n)$,
luôn được sắp.

GHI NHỚ · KEY POINT

`TreeSet` cho thêm truy vấn theo khoảng: `first`, `last`, `ceiling`, `floor`. `Set` không cho phần tử trùng.

Set: khác biệt thứ tự duyệt *Iteration order*

DAU

```java
Set<String> a = new HashSet<>(); // no order guarantee, O(1) average
a.add("b"); a.add("a"); a.add("b"); // duplicate "b" ignored
System.out.println(a.size()); // 2

Set<String> b = new LinkedHashSet<>(); // preserves insertion order
b.add("b"); b.add("a"); b.add("c"); // iterates as b, a, c

Set<String> c = new TreeSet<>(); // sorted order, O(log n)
c.add("b"); c.add("a"); c.add("c"); // iterates as a, b, c
System.out.println(((TreeSet<String>) c).first()); // "a"
```



5

Ánh xạ

HashMap, TreeMap

Map: cặp khoá-giá trị *Key-value pairs*

DAU

ĐỊNH NGHĨA · DEFINITION

`Map` lưu các cặp khoá-giá trị, khoá không trùng. Cấu trúc tra cứu quan trọng bậc nhất: từ điển, bộ đếm tần suất, bộ nhớ đệm.

- `HashMap`: tra cứu trung bình $O(1)$, không bảo đảm thứ tự.
- `LinkedHashMap`: duyệt theo thứ tự chèn.
- `TreeMap`: khoá luôn được sắp, thao tác $O(\log n)$.

Bộ ba cài đặt của `Map` phản chiếu đúng bộ ba của `Set`.

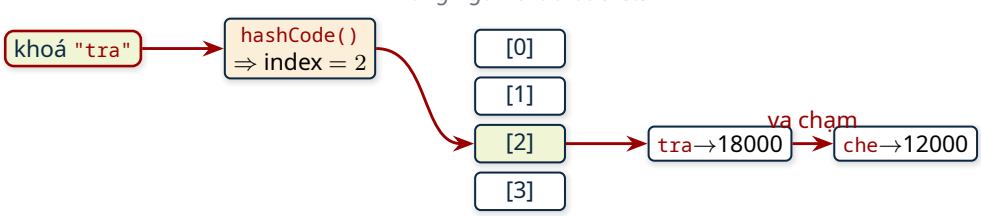
Map: thao tác cốt lõi và đếm tần suất *Core ops*

DAU

```java
Map<String, Integer> gia = new HashMap<>();
gia.put("ca phe", 25000); // insert key-value, O(1) average
gia.put("tra", 18000);
int g = gia.getOrDefault("nuoc", 0); // 0 if key absent, no exception

String[] tu = {"a", "b", "a", "c", "a"};
Map<String, Integer> dem = new HashMap<>();
for (String t : tu) {
   dem.merge(t, 1, Integer::sum); // add 1, or start at 1 if absent
}
// dem = {a=3, b=1, c=1}
```

Bên trong HashMap: ngăn chứa và va chạm *Buckets and collisions*



Khoá được băm về một ngăn; các khoá trùng ngăn nối thành chuỗi, ngăn dài chuyển thành cây từ Java 8.



6

equals & hashCode

contract for hashing

Hợp đồng equals-hashCode *The contract*

ĐỊNH NGHĨA · DEFINITION

Nếu `a.equals(b)` đúng thì `a.hashCode()` phải bằng `b.hashCode()`. Chiều ngược lại không bắt buộc: cùng `hashCode` vẫn có thể khác nhau.

- `equals`: hai đối tượng có được coi là bằng nhau hay không.
- `hashCode`: đối tượng thuộc ngăn nào.

LỖI HAY GẶP · PITFALL

Ghi đè `equals` mà quên `hashCode`: hai đối tượng bằng nhau rơi vào hai ngăn khác, `HashSet` chứa cả hai. Luôn ghi đè cả hai cùng lúc.

Ghi đè equals và hashCode *Override both*

DAU

```java
public final class Diem {
   private final int x, y;
   public Diem(int x, int y) { this.x = x; this.y = y; }

   @Override
   public boolean equals(Object o) {
      if (this == o) return true; // same reference
      if (!(o instanceof Diem d)) return false; // type check + pattern
      return x == d.x && y == d.y; // compare same fields
   }
   @Override
   public int hashCode() {
      return Objects.hash(x, y); // must use the same fields as equals
   }
}
```

record sinh sẵn equals-hashCode *Records*

GHI NHỚ · KEY POINT

Từ Java 16, lớp `record` sinh sẵn `equals`, `hashCode` và `toString` theo các trường thành phần, nên hiếm khi phải tự viết. Ta gặp `record` ở chương 6.

Dù vậy, hiểu hợp đồng vẫn cần thiết để biết khi nào tự cài là an toàn, ví dụ khi lớp không phải `record` hoặc cần định nghĩa bằng nhau theo tập trường con.



7

Sắp xếp

Comparable & Comparator

Hai con đường sắp xếp *Two ways to order*

DAU

`Comparable`
thứ tự tự nhiên gắn vào lớp qua `compareTo`.
Mỗi lớp chỉ một thứ tự tự nhiên.

`Comparator`
đối tượng so sánh bên ngoài.
Nhiều tiêu chí, không sửa lớp gốc.

So sánh trả số âm nếu đứng trước, dương nếu đứng sau, 0 nếu tương đương. Từ Java 8 có `comparing`, `thenComparing`, `reversed` để ghép tiêu chí.

Comparable và Comparator trong mã *In code*

DAU

```java
public class SinhVien implements Comparable<SinhVien> {
   String ten; double diem;
   SinhVien(String t, double d) { ten = t; diem = d; }
   @Override
   public int compareTo(SinhVien o) { // natural order: by name
      return this.ten.compareTo(o.ten);
   }
}
// Multi-key: score descending, then name ascending
ds.sort(Comparator.comparingDouble((SinhVien s) -> s.diem).reversed()
               .thenComparing(s -> s.ten));
// A TreeSet can take a Comparator to override natural order
Set<SinhVien> theoDiem =
   new TreeSet<>(Comparator.comparingDouble(s -> s.diem));
```

Khi nào dùng cái nào *When to use which*

DAU

GHI NHỚ · KEY POINT

Dùng `Comparable` cho thứ tự duy nhất và hiển nhiên của một lớp.

Dùng `Comparator` khi cần nhiều cách sắp, khi không sửa được lớp gốc, hoặc khi muốn sắp khác với thứ tự tự nhiên.



8

Chọn cấu trúc

performance & Big-O

Bảng độ phức tạp trung bình *Big-O summary*

DAU

Cấu trúc	Truy cập	Tìm	Thêm	Xoá	Đặc điểm
`ArrayList`	$O(1)$	$O(n)$	$O(1)$*	$O(n)$	có chỉ số, duyệt nhanh
`LinkedList`	$O(n)$	$O(n)$	$O(1)$	$O(1)$	thêm xoá ở hai đầu
`ArrayDeque`	–	–	$O(1)$	$O(1)$	hàng đợi, ngăn xếp
`PriorityQueue`	–	–	$O(\log n)$	$O(\log n)$	lấy nhỏ nhất trước
`HashSet`	–	$O(1)$	$O(1)$	$O(1)$	không thứ tự, loại trùng
`LinkedHashSet`	–	$O(1)$	$O(1)$	$O(1)$	giữ thứ tự chèn
`TreeSet`	–	$O(\log n)$	$O(\log n)$	$O(\log n)$	luôn được sắp
`HashMap`	–	$O(1)$	$O(1)$	$O(1)$	tra khoá nhanh
`LinkedHashMap`	–	$O(1)$	$O(1)$	$O(1)$	giữ thứ tự chèn
`TreeMap`	–	$O(\log n)$	$O(\log n)$	$O(\log n)$	khoá luôn được sắp

* Thêm vào cuối `ArrayList` là $O(1)$ khấu hao *amortized*; thỉnh thoảng $O(n)$ khi tăng dung lượng.

Quy tắc chọn nhanh *Rules of thumb*

Truy cập chỉ số, duyệt tuần tự → `ArrayList`.
Thêm xoá hai đầu → `ArrayDeque`.
Loại trùng, tra nhanh → `HashSet`.

Tra khoá-giá trị nhanh → `HashMap`.
Giữ luôn được sắp → `TreeSet`/`TreeMap`.
Giữ thứ tự chèn → thêm tiền tố `Linked`.

GHI NHỚ · KEY POINT

Chọn theo thao tác chiếm đa số trong bài toán.

Ví dụ: gom nhóm và đếm bằng đúng cấu trúc *Worked example*

DAU

```java
String[] luot = {"tra", "ca phe", "tra", "banh", "ca phe", "tra"};
Map<String, Integer> dem = new TreeMap<>(); // sorted by key
for (String m : luot) {
   dem.merge(m, 1, Integer::sum); // count occurrences
}
dem.forEach((k, v) -> System.out.println(k + ": " + v));
// banh: 1
// ca phe: 2
// tra: 3
```

Cần kết quả sắp theo khoá nên dùng `TreeMap`. Nếu không cần thứ tự, đổi sang `HashMap` để tra nhanh hơn: chỉ sửa một dòng khởi tạo nhờ lập trình theo giao tiếp.

9

Tóm tắt & bài tập

summary & exercises

Tóm tắt chương *Summary*

DAU

Hai nhánh: `Collection` và `Map` tách rời.

`List`, `Queue`/`Deque`, `Set`, `Map` mỗi loại nhiều cài đặt.

Bảng băm cần `equals`/`hashCode` đúng.

Sắp xếp bằng `Comparable`/`Comparator`.

GHI NHỚ · KEY POINT

Lập trình theo giao tiếp: khai báo `List`, `Set`, `Map`. Chọn cấu trúc theo Big-O của thao tác chiếm đa số.

Bài tập về nhà *Homework*

1. Giải thích vì sao `Map` không kế thừa `Collection`: một lý do mô hình dữ liệu, một hệ quả thực tế.
2. Chọn cấu trúc cho: (a) số điện thoại không trùng in tăng dần; (b) hàng chờ công việc theo thứ tự đến; (c) bảng tra điểm theo mã sinh viên; (d) ngăn xếp hoàn tác *undo*.
3. Quên `hashCode` khi đã ghi đè `equals`: mô tả chính xác điều xảy ra khi đặt hai `Diem` bằng nhau vào `HashSet`.
4. Viết `TanSoTu`: đếm tần suất từ bằng `HashMap`, in mười từ nhiều nhất theo tần suất giảm dần.
5. Dùng `PriorityQueue` trộn k danh sách đã sắp thành một danh sách sắp tăng; phân tích độ phức tạp.

CẢM ƠN

Hỏi đáp · Thảo luận

TS. Đỗ Phúc Hảo

haodp.ai@gmail.com · 0365 674 592 · Khoa CNTT, ĐH Kiến trúc Đà Nẵng